AR Indoor Navigation R&D Report
Android Approach and Integration Guide

Purpose

This document explains how indoor navigation will work in our project, why we chose an ARCore-based visual mapping approach with printed AR signs, and why we are not using BLE beacons, Wi-Fi RTT, UWB, Navigine or ARway Kit. It also lists the rules the users who record the paths must follow so that navigation works reliably for visitors.

The core idea of the project is simple: help visitors find their way to destinations inside buildings in Qatar (museums, malls, exhibition halls, airports, hotels and similar venues) using augmented reality on their phone.

R&D journey and struggles

We first tried the suggested and some other third-party SDKs (like Navigine, ARway Kit, Immersal).

SDK	What we did	Result
Navigine	Navigine needs a floor map, so we drew a custom floor plan of the test area. We mapped the plan, walls and routes in their portal and placed beacons.	Their app did not detect the beacons we placed. At times it still showed old beacons on the map that had been removed an hour earlier. AR indoor navigation is listed as one of their capabilities, but we could not find it in their app.
ARway Kit	Reviewed the SDK.	Not purely native, so it was ruled out.
Immersal	Reviewed the SDK and its sample code.	The code we found was outdated.

Goals of this R&D

The R&D aimed to prove the core of the visitor journey in the requirements:

- Pick a destination and launch AR navigation
  - The phone can find its position inside a building and draw arrows to a destination
- Arrows, labels and distance in the camera view
  - AR content stays fixed to the real floor and walls while the user walks
- Turn-by-turn guidance
  - Off-route detection and rerouting
- Contextual tips and arrival confirmation
  - Location-based tips and an arrival screen
- Offline AR map caching
  - The venue can be downloaded once with network, and used without network

How the WayFinder application was built

This section lists the tools and libraries we used, the architecture we followed, and the steps we took to build the prototype.

Tools

Tool	Used for
Android Studio and Gradle (Kotlin DSL)	Building and running the app, with one version catalog (libs.versions.toml) for all library versions.
Android Debug Bridge (adb)	Installing the debug build on the phones, reading logs, and copying path files to and from a phone during testing.
Real ARCore phones	Samsung Galaxy A54 and CPH2661 (both Android 16). Camera tracking cannot be judged on an emulator.
JUnit	Automated unit tests of the navigation logic (routing, alignment, recording, storage, arrival).
Printed AR signs	Images in the venue package, printed at their exact size, used so the phone knows where it is.

Libraries

Library	Version	Purpose
Kotlin	2.4.21	The only language used. Android Gradle Plugin 9.4.1.
ARCore (com.google.ar:core)	1.57.0	Motion tracking, Augmented Images (sign detection), planes and depth, anchors, hit tests.
SceneView (arsceneview)	4.60.0	The AR camera view and 3D drawing (Filament renderer): arrows, pin, objects and name labels.
Jetpack Compose and Material 3	BOM 2026.02.01	All screens: destination list, navigation overlay, recording controls, dialogs.
kotlinx.serialization	1.12.0	Reading venue.json and reading and writing the path files (JSON).
AndroidX Core, Lifecycle, Activity Compose	see catalog	Standard Android support libraries.

Architecture

We followed a layered, clean-style structure: the navigation rules live in a core layer that knows nothing about Android, the AR and screen code sits above it, and the app is a thin shell on top. Dependencies point only one way (app, then AR layer, then core), so the core can be tested on a computer in seconds and reused in another app.

Layer	Module	What it holds
App	app	The activity, the destination list, "Add destination", camera permission, and where path files are saved on the phone.

Layer	Module	What it holds
AR and screens	wayfinder-ar	ARCore session and sign tracking, the navigation screen and the recording screen, 3D arrows, pin and objects, the on-screen HUD.
Core (pure Kotlin)	wayfinder-core	Venue model, A* routing over the walked path, turn instructions, sign alignment maths and drift filter, arrival check, path recorder, path file storage. No Android code.

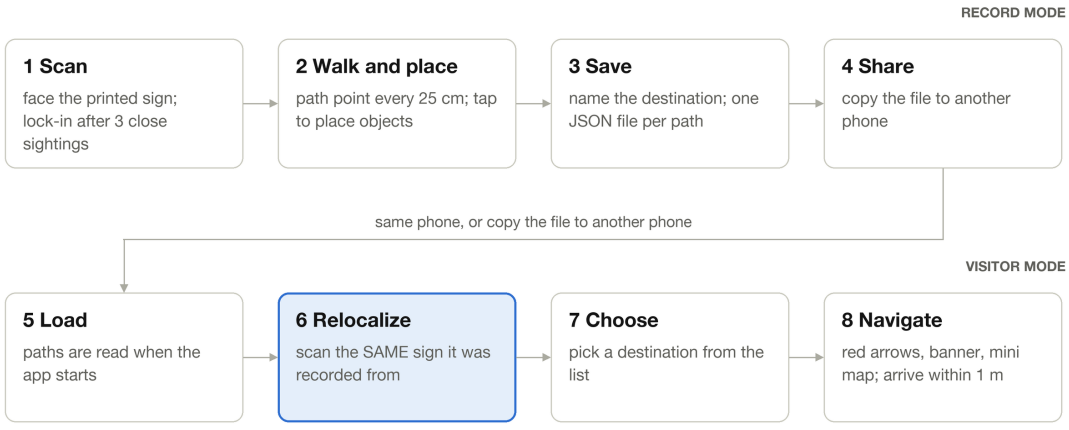
Note: we did not use a dependency-injection framework or a full use-case and repository split; the prototype is small, so plain classes were enough. These are natural next steps for a production app.

The prototype, step by step

1. User: open the app, tap "Add destination", scan the printed sign until the app says it is locked in.
2. User: walk to the destination. The app records the route on the floor as they walk.
3. User: tap the screen to place objects along the way and type a name for each. Then hold steady and save. The destination is set where the user stands.
4. Visitor: open the app and choose the destination from the list.
5. Visitor: scan the same printed sign. The app aligns the venue to the real world.
6. Visitor: follow the red arrows along the recorded route. The objects and their names appear along the way, with a mini map and a banner showing the distance left.
7. Arrival: when the visitor reaches the end of the route and stays for a second, the app shows that they have arrived.

Workflow of the Android prototype

A user records a path once; visitors replay it after the app recognises the printed sign.



Venue data and file format

In this R&D all data is stored offline on the phone, as plain JSON files and sign images. There is no backend and no network call: the app works without a network. Distances are in metres, measured from the printed sign.

File	Contents
venue.json (inside the app)	The venue description: levels, destinations, and the printed signs (id, image name, printed width, position, direction and mounting). Read-only; it ships with the app.
markers/<sign>.png (inside the app)	The image of each printed sign. ARCore loads these to recognise the sign in the camera view.
paths/<destination-name>.json (saved on the phone)	One file per recorded path, created when the user saves. For example printer.json. If the name is already used, -2, -3 and so on is added. Stored in the app's private storage, and can be copied to another phone.

What a path file contains:

Field	Meaning
destination	The destination: id, name, level, position (x, z), floor height and the time it was created. It is set automatically where the user finishes the walk.
walkedPoints	The recorded route: a numbered list of floor points (x, y, z) in walking order. Routing and the arrows follow these points.
objects	The 3D objects placed along the path: type (for example cup, gift, star), name shown above it, position (x, y, z), direction and time. The reference object at the sign is saved here too.
startMarkerId	The sign the path was recorded from. Scan the same sign to follow the path.
alignmentSpreadCm, driftCorrectedCm	How well the sign scans agreed while recording, and how much drift a re-scan corrected. Used to judge accuracy.
venueId, formatVersion	The venue the path belongs to, and the file format version.

A path file can be copied to another Android phone, which then shows the same destination and route after the user scans the same printed sign. Uploading and downloading files through an API is not part of this R&D.

Mapping guidelines (must follow)

The quality of navigation depends entirely on the quality of mapping. Users who record paths must follow these rules:

- **Map the most suitable routes:** While recording, follow clear, practical and accessible paths, keeping a safe distance from walls and obstacles, so navigation is smooth and reliable for visitors.
- **Ensure proper route connectivity:** Record all the necessary paths that connect different areas and destinations, so navigation works throughout the mapped venue.
- **Place printed signs at strategic locations:** Place signs at as many suitable locations as practical, such as building entrances and exits and key junctions, so visitors can easily find a sign to scan and align their position. Each path is aligned from the one sign it was recorded from.
- **Multiple floors (stairs and lifts):** Not supported in this R&D. AR tracking becomes unreliable in lifts and on stairs. Map each floor separately, from a sign on that floor, and never record a path through a lift or stairs.

- **Maintain adequate lighting:** Record in well-lit places. Low light affects ARCore's ability to recognise the surroundings and the printed sign, and to keep tracking accurate.
- **Consider environmental features:** Areas with distinctive visual features and textures support better AR tracking. Take extra care in areas with plain walls, glass surfaces, mirrors or highly reflective flooring.
- **Maintain consistent signs:** Signs should stay in a fixed position with a consistent look and direction. Use permanent, high-contrast printed signs at their exact declared size, with a non-reflective matte finish, mounted flat, to improve recognition reliability.
- **Maintain steady movement while recording:** Walk at a comfortable, consistent pace, keep the phone stable, and avoid sudden movements and sharp turns so the recorded path is accurate. Hold the phone steady for a moment when saving.
- **Update paths when necessary:** Significant changes to the venue layout, furniture, signage or lighting may affect navigation accuracy. In such cases, review the affected areas and record the paths again where needed, then replace the saved path files.

Limitations of this R&D

This was a quick R&D. It is a short proof of concept to find out whether ARCore plus printed signs can guide a visitor inside a building. It is not a finished, production-ready product. The limitations below are known and should be considered before planning a rollout.

Limitation	What happens	What to do for now
Multiple floors and lifts are not supported	A path stays on one floor. A lift ride gives ARCore nothing steady to follow, so tracking and heights after the ride are unreliable. A path recorded across floors is saved as if it were flat, and the destination appears at the wrong height and place.	Map each floor separately, from a sign on that floor. Never record a path through a lift or stairs.
Dim places	ARCore follows the camera image. In very dim light it cannot find enough detail, so tracking does not start or is lost, and the printed sign cannot be recognised.	Use it only where the area is reasonably well lit. Avoid dark corridors and dark rooms.
Very long distances	ARCore drifts by a few centimetres as the phone moves, and the error grows over a long walk. A path is aligned from one sign, so the far end of a long route can be noticeably off.	Keep each path short, or split a long route into several paths with a sign at the start of each. Re-scan a sign during the walk to correct drift.

Summary

This quick R&D built a native Android prototype with ARCore and printed AR signs. A user scans a sign, walks to a destination, places named 3D objects and saves the path as a JSON file. A visitor scans the same sign, chooses the destination and follows red arrows along the recorded route, with a mini map and an arrival check. Paths are stored offline on the phone and can be copied to another Android phone.

The approach worked in a small, well-lit area on two phones. It does not support lifts or multiple floors, may fail in very dim light, and loses accuracy over long distances. We have not yet measured the position error in centimetres, so accuracy needs more testing in a real venue before any rollout.

Demo

Video Reference: Android_AR_Navigation_Demo.mp4